

DRC PAY

How the system is built & how a payment flows

A Python/FastAPI backend runs the payment logic against a double-entry ledger in PostgreSQL and answers requests over a REST API, while **pawaPay** actually moves the mobile money — we stay a pure pass-through and never hold funds. The screens are plain web pages; the whole thing ships as a Docker container on Railway.

Two infographics · technical components, plain-English captions · 2026-06-19

INFOGRAPHIC 1

Application architecture

Top to bottom: the screens people use, the backend that holds the logic, the systems it depends on, and where it runs. A request travels down; the answer travels back up.

FRONT-END — WHAT PEOPLE USE (NO APP TO INSTALL)

- **Merchant Console**

static HTML / CSS / JS

The shopkeeper's dashboard: create a charge QR, watch payments confirm live, drill into the ledger.

- **Customer Pay Page**

static HTML / CSS / JS

Opens when a customer scans the QR — shows the locked amount, pick a network, pay. No login.

- **USSD channel**

/ussd · provider-neutral

Feature phones with no internet dial a code — the same backend, a different door.

↓ HTTP · JSON over the internet ↓

Back-end — Python · FastAPI · REST API

Layered so the money rules sit in a pure core that knows nothing about the web or the database.

Channels — the "doors"

`http/ · merchant_api · public · webhook · ussd · demo`

Translate each incoming request into Python, check it, and hand it off. No money logic here.



Application services

`start_merchant_payment · apply_outcome`

One shared entry point every channel calls — resolves operators, computes the fee, starts the work.



Domain — the money brain (pure)

`Orchestrator · double-entry ledger · state machine · pricing`

The rules of money: collect → settle → auto-refund, every posting balanced. No HTTP, no SQL — this is what the tests cover most.



Adapters — ports to real infrastructure

`SQLAlchemy stores · pawaPay rail / simulator`

Swappable plugs: real Postgres & pawaPay in production; in-memory store & a simulator in tests (so they run offline).

Dependencies point inward — swap the adapters (simulator ↔ pawaPay, memory ↔ Postgres) without touching the core.

↓ reads & writes ↓

THE SYSTEMS THE BACKEND DEPENDS ON

- **PostgreSQL — the books & state**

SQLAlchemy + Alembic migrations

Stores the transactions and the double-entry ledger — the real source of truth for money. (In-memory stand-in for dev & tests.)

- **pawaPay — mobile-money aggregator**

Vodacom M-Pesa · Airtel · Orange

Actually moves the funds at the operators; we never hold money (pure pass-through). Confirms each step with a signed (RFC-9421) webhook.

☁ **Runs as** one Docker container on Railway — a single image serves the API + both web pages; eventual home is AWS (af-south-1).

■ front-end & backend (our code) ■ the pure money core ■ database ■ where it runs

INFOGRAPHIC 2

Payment flow — scan to paid

One successful payment, end to end. The customer needs no app — they just scan the merchant's charge QR. Steps in **green** are where money actually moves and the books are written.

1

Customer scans the charge QR

QR → /customer/?charge=<id>

The QR encodes a link to the pay page for this exact charge. Their phone's browser opens it — nothing to install.

2

The pay page loads the charge

GET /public/charge/{id}

Shows the locked amount and the merchant. The amount is set by the server from the charge — never typed by the customer.

3

Customer picks their network & taps Pay

POST /pay

Their mobile-money operator (Vodacom / Airtel / Orange) plus the charge's amount go to the backend.

4

The backend starts the payment

start_merchant_payment → Orchestrator

Computes the fee from the network pair, creates the transaction, and kicks off the state machine.

5

Collect **MONEY IN FLIGHT**

pawaPay deposit

pawaPay debits the customer; the funds sit at pawaPay (a "clearing" holding spot). Nothing is settled yet.

6

Settle to the merchant **BOOKS WRITTEN**

pawaPay payout · ledger postings

The merchant receives **amount – fee**; pawaPay's cost is booked to **expense**, our margin to **revenue**. Every posting balances.

7

Confirm — both screens go "Paid"

`signed webhook (RFC-9421)`

pawaPay confirms each leg with a signed callback; the Merchant Console and the customer's page both flip to **Paid** in real time.

↪ If the payout fails after collection — the auto-refund branch

The customer is automatically refunded the **full amount**. pawaPay already took its collection fee, so that is booked as a **real loss** (`expense:pawapay`) — visible on the books, never hidden. A refund that itself fails is escalated to `manual_review`. If the customer's callback never arrives, a reconciliation sweep polls pawaPay and heals it.

INFOGRAPHIC 3

Where the money goes — the ledger

Every payment is recorded as balanced double-entry postings — the real source of truth for money. Here's one \$10 payment (customer on Vodacom, merchant settles to Airtel), dollar by dollar.

Customer pays \$10.00

↓ splits three ways ↓

\$9.55

Merchant receives

`merchant:external`

The sticker amount minus the fee — paid out to their mobile money.

\$0.45

pawaPay's cost

`expense:pawapay`

The rails fee — collect 0.25 (Vodacom) + payout 0.20 (Airtel) — booked as an expense.

\$0.00

We keep

`revenue:fees`

Our margin — zero today, because the fee only covers cost (no markup yet).

merchant — \$9.55

■ merchant 9.55 ■ pawaPay cost 0.45 ■ our margin 0.00

THE ACTUAL DOUBLE-ENTRY POSTINGS — EVERY ONE BALANCES (DEBITS = CREDITS)

1 • Collection succeeds

debit	customer:external	10.00
credit	pawapay:clearing	9.75
credit	expense:pawapay	0.25
✓ balanced		10.00 = 10.00

2 • Settlement to the merchant

debit	pawapay:clearing	9.75
credit	merchant:external	9.55
credit	expense:pawapay	0.20
✓ balanced		9.75 = 9.75

+ The seam where profit appears

Today the fee (MDR) equals pawaPay's cost, so `revenue:fees` is \$0. Mark the MDR up — say to 6% = \$0.60 — and the merchant nets \$9.40, pawaPay still costs \$0.45, and the extra **\$0.15 lands in revenue automatically**. That one number is the pricing decision.

↪ What a refund looks like on the books

If the payout fails after collection, the customer is refunded the full **\$10.00** — but pawaPay already took its **\$0.25** collection fee, so that stays in `expense:pawapay` as a booked loss. The books always show exactly where every dollar went.

DRC Pay · merchant-facing mobile-money acceptance on rented rails · pure pass-through (no fund custody)
Self-contained — open this file in any browser. Brand-coloured to match the Merchant Console.